

Smart Lender – Loan Prediction System

Project Description:

Smart Lender – Loan Prediction System is a Machine Learning-based web application that predicts whether a loan applicant is eligible for loan approval based on financial and personal details. The application uses Flask for the backend, HTML/CSS/JavaScript for the frontend, and a trained Scikit-learn model for prediction.

Scenarios

Scenario 1

A salaried applicant enters all required details with a good credit history. The system predicts that the loan is approved.

Scenario 2

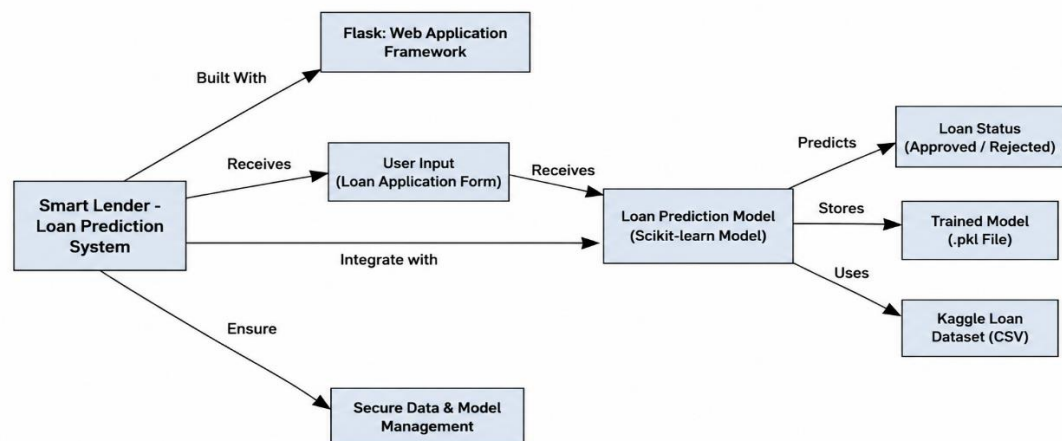
A self-employed applicant with low income and poor credit history submits details. The system predicts that the loan is not approved.

Scenario 3

A bank officer uses the application to quickly verify multiple applicants' loan eligibility.

Technical Architecture:

Technical Architecture:



Description: CaptionAI utilizes a modular three-tier architecture to deliver rapid, AI-driven content generation. The frontend provides a responsive user interface built with HTML and CSS while the Flask-based backend orchestrates data validation and prompt construction. It interfaces with the Groq API using the LLaMA 3.3-70B model to generate tailored social media content, with secure deployment managed locally via Flask and exposed publicly through Ngrok.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

Flask Framework Knowledge: Flask Documentation

Scikit-learn Knowledge: Scikit-learn Documentation

HTML, CSS, and JavaScript Skills: W3Schools HTML/CSS/JavaScript Tutorials

Python Programming Proficiency: Python Documentation

Pandas & NumPy Knowledge: Pandas and NumPy Documentation

Version Control with Git: Git Documentation

Development Environment Setup: Flask Installation Guide

Machine Learning Basics: Kaggle Learn – Intro to Machine Learning

Project Workflow:

Milestone 1: Dataset Preparation and Model Development

- **Activity 1.1:** Collect the Loan Prediction dataset from Kaggle.
- **Activity 1.2:** Perform data preprocessing, including handling missing values, encoding categorical features, and feature scaling.
- **Activity 1.3:** Train multiple Machine Learning models (Decision Tree, Random Forest, KNN, and XGBoost).
- **Activity 1.4:** Select the best-performing model based on evaluation metrics and save it as a .pkl file

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the loan eligibility prediction module using the trained Machine Learning model.
- **Activity 2.2:** Implement Flask backend routes to receive applicant details and generate prediction results.
- **Activity 2.3:** Validate user inputs and display loan approval or rejection results.

Milestone 3: App.py Development

- **Activity 3.1:** Develop the main application logic in app.py.
- **Activity 3.2:** Create routes for the Home page and Prediction page.
- **Activity 3.3:** Load the trained Machine Learning model and process prediction requests.

Milestone 4: Frontend Development

- **Activity 4.1:** Design the Loan Application Form using HTML, CSS, Bootstrap, and JavaScript.
- **Activity 4.2:** Create responsive web pages using Flask templates.
- **Activity 4.3:** Display prediction results in a user-friendly interface.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the application using different loan applicant records.
- **Activity 5.2:** Fix bugs and validate prediction accuracy.
- **Activity 5.3:** Deploy and run the application on the Local Flask Server.

Milestone 6: Conclusion

- **Activity 6.1:** Evaluate the application's performance and prediction accuracy.
- **Activity 6.2:** Document project outcomes and identify future enhancements.

Milestone 1: Dataset Collection and Model Architecture

Activity 1.1: Download Loan Dataset

- Visit the Kaggle website and search for the Loan Prediction Dataset.
- Download the dataset in CSV format.
- Store the dataset inside the Dataset folder.
- Verify that all required columns such as Applicant Income, Loan Amount, Credit History, and Loan Status are present.

 Search

Loan Prediction Problem Dataset

[Data Card](#)
[Code \(269\)](#)
[Discussion \(9\)](#)
[Suggestions \(1\)](#)

About Dataset

No description available

Usability ⓘ
1.76

License
Unknown

Expected update frequency
Not specified

Tags

Activity 1.2: Prepare the Development Environment

- Install Python 3.10 or later.
- Install all required libraries using:

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL>pip install flask pandas numpy scikit-learn joblib
Requirement already satisfied: flask in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (3.1.3)
Requirement already satisfied: pandas in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (3.0.3)
Requirement already satisfied: numpy in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (2.5.1)
Requirement already satisfied: scikit-learn in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (1.9.0)
Requirement already satisfied: joblib in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (1.5.3)
Requirement already satisfied: blinker>=1.9.0 in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (from flask) (8.1.3)
Requirement already satisfied: itsdangerous>=2.2.0 in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (from flask) (2.2.0)
Requirement already satisfied: Jinja2>=3.1.2 in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (from flask) (3.1.2)
Requirement already satisfied: MarkupSafe>=2.1.1 in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (from Jinja2) (2.1.1)
Requirement already satisfied: Werkzeug>=3.1.0 in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (from flask) (3.1.0)
Requirement already satisfied: python-dateutil>=2.8.2 in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pandas) (2.8.2)
Requirement already satisfied: tzdata in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pandas) (2024.1)
Requirement already satisfied: scipy>=1.10.0 in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (from scikit-learn) (1.10.0)
Requirement already satisfied: narwhals>=2.0.1 in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pandas) (2.0.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in .\AppData\Local\Programs\Python\Python312\Lib\site-packages (from scikit-learn) (3.5.0)
```

- . Verify the installation using:

```
C:\Users\DELL>python --version
Python 3.12.1

C:\Users\DELL>pip list
Package Version
-----
blinker 1.9.0
click 8.4.2
colorama 0.4.6
contourpy 1.3.3
cycler 0.12.1
Flask 3.1.3
fonttools 4.63.0
itsdangerous 2.2.0
Jinja2 3.1.2
joblib 1.5.3
kiwisolver 1.5.0
MarkupSafe 3.0.3
matplotlib 3.11.0
narwhals 2.23.0
numpy 2.5.1
packaging 26.2
pandas 3.0.3
pillow 12.3.0
pip 26.1.2
pyparsing 3.3.2
python-dateutil 2.9.0.post0
scikit-learn 1.9.0
scipy 1.18.0
```

Activity 1.3: Train the Machine Learning Model

- Open model_training.py.
- Load the dataset using pandas.
- Preprocess missing values.
- Encode categorical variables.
- Split the dataset into training and testing sets.
- Train the Random Forest classifier.
- Save the trained model as loan_model.pkl.

Activity 1.4: Create the Flask Application

- Develop the Flask backend using app.py.
- Load the trained model.
- Accept user input from the web form.
- Predict loan eligibility.
- Display the prediction on the web page.

```

13
14 @app.route("/")
15 def home():
16     return render_template("index.html")
17
18
19
20 @app.route("/predict", methods=["POST"])
21 def predict():
22     Gender = float(request.form["Gender"])
23     Married = float(request.form["Married"])
24     Dependents = float(request.form["Dependents"])
25     Education = float(request.form["Education"])
26     Self_Employed = float(request.form["Self_Employed"])
27     ApplicantIncome = float(request.form["ApplicantIncome"])
28
29
30

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 586-829-724

```

Activity 1.5: Design the User Interface

- Create responsive pages using HTML and CSS.
- Collect applicant details through the loan application form.
- Display prediction results clearly.

Smart Lender - Loan Pre

Gender (Male=1, Female=0)

Married (Yes=1, No=0)

Dependents

Education (Graduate=1, Not Graduate=0)

Self Employed (Yes=1, No=0)

Applicant Income

Coapplicant Income

Loan Amount

Milestone 2: Core Functionalities Development

Activity 2.1: Functional Testing

- Verify all form fields.
- Check prediction accuracy.
- Validate invalid inputs.

Activity 2.2: Performance Testing

- Measure response time.
- Test multiple requests.
- Ensure the application performs correctly under

Milestone 3: Deployment

Activity 3.1: Upload Project to GitHub

- Create a GitHub repository.
- Push the source code.
- Verify all folders are uploaded.

Home Route

```
@app.route("/")
def index():
    return render_template("index.html")@app.route("/generate", methods=["POST"])
```

The home route loads the Smart-Lender user interface where users can enter loan applicant details.

Prediction Route

```
@app.route("/predict", methods=["POST"])
def predict(): predict", methods=["POST"])
```

This route receives the user input, validates it, loads the trained machine learning model, predicts loan eligibility, and displays the result.

Input Validation

- Accept applicant information from the HTML form.
- Check whether mandatory fields are provided.

- Convert numerical values into appropriate data types.
- Prevent invalid or empty submissions before prediction.

Load Machine Learning Model

```
model = joblib.load("Models/loan_prediction_model.pkl")
```

The trained Random Forest model is loaded from the Models folder.

Generate Prediction

```
prediction = model.predict(features)
```

The model predicts whether the loan should be **Approved** or **Rejected**

Activity 3.2: Backend Processing

The Flask backend handles all prediction requests.

It performs the following tasks:

- Receives applicant details.
- Performs input validation.
- Loads the trained ML model.
- Predicts loan approval.
- Sends prediction results back to the user interface.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a simple, responsive, and user-friendly interface for the **Smart-Lender Loan Approval Prediction System**. The frontend is designed using **HTML, CSS, and Bootstrap**, allowing users to enter loan applicant details easily and receive instant loan eligibility predictions generated by the Machine Learning model.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

- Develop the main HTML page (**index.html**) containing the application title, loan prediction form, and prediction result section.
- Use semantic HTML elements to organize the webpage and improve readability.
- Create separate sections for applicant information and prediction output.
- Add proper labels, placeholders, and validation messages for every input field.

Design a Responsive Layout Using CSS & Bootstrap

- Use **Bootstrap 5** to build a clean and responsive layout.
- Create a centered prediction form using Bootstrap Grid.
- Apply CSS styling for buttons, cards, spacing, fonts, and colors.
- Ensure the webpage is compatible with desktop, tablet, and mobile devices.

Create Input Components for Loan Prediction

Develop separate input fields for all loan applicant information:

- Gender
- Married Status
- Education
- Self Employed
- Applicant Income
- Co-Applicant Income
- Loan Amount
- Loan Amount Term

- Credit History
- Property Area

Create Prediction Result Section

- Display the prediction result after form submission.
- Show:
 - **Loan Approved**
 - **Loan Rejected**
- Highlight the result using Bootstrap Alert components.
- Display prediction instantly without refreshing the entire application.

Activity 4.2: Creating Dynamic Interaction with Flask

Handle Form Submission

- Submit loan applicant details using an HTML form.
- Send all user inputs securely to the Flask backend using the **POST** method.
- Validate required fields before submitting the form.

Receive Prediction from Flask

- Flask receives the form data.
- Converts user input into numerical values required by the trained Machine Learning model.
- Passes the processed data to the trained Scikit-Learn model.
- Receives the prediction result.

Display Prediction Result

- Return the prediction result from Flask back to the HTML page.
- Display the result immediately inside the prediction section.
- Show success or rejection messages in a user-friendly format.

Improve User Experience

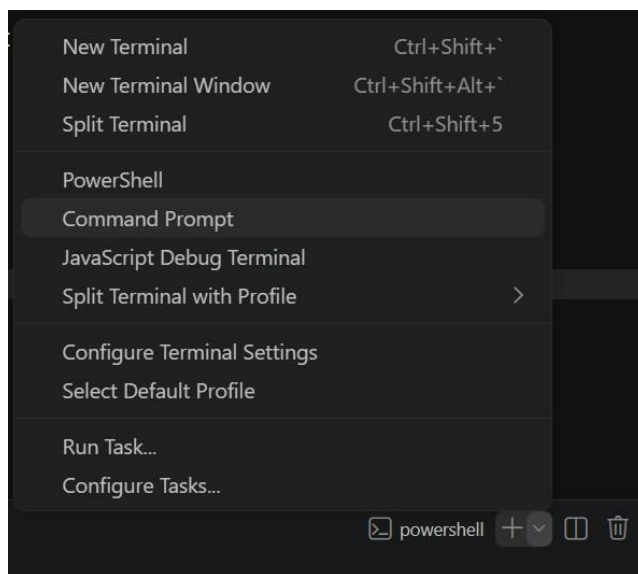
- Validate user inputs before prediction.
- Prevent empty form submission.
- Display meaningful error messages for invalid inputs.
- Keep the interface simple, responsive, and easy to use.

Technologies Used

- HTML5
- CSS3
- Bootstrap 5
- Flask (Python)
- Jinja2 Templates
- JavaScript (Basic Form Validation)
- **Milestone 5: Deployment**
 - In Milestone 5, the Smart-Lender application is prepared for execution and demonstration. The application is deployed locally using the Flask Development Server to verify that all functionalities work correctly. The project source code is managed using GitHub for version control and collaboration.
- **Activity 5.1: Preparing the Application for Local Deployment**
- **Open a New Terminal**

```
C:\Projects\Smart-Lender> python -m venv venv
```

- Open **Visual Studio Code**.
- Open the Smart-Lender project folder.
- Open **Terminal** → **New Terminal** or Command Prompt



- Create and activate a virtual environment, then install all required dependencies from requirements.

Activate the virtual environment.

```
C:\Projects\Smart-Lender> venv\Scripts\activate
```

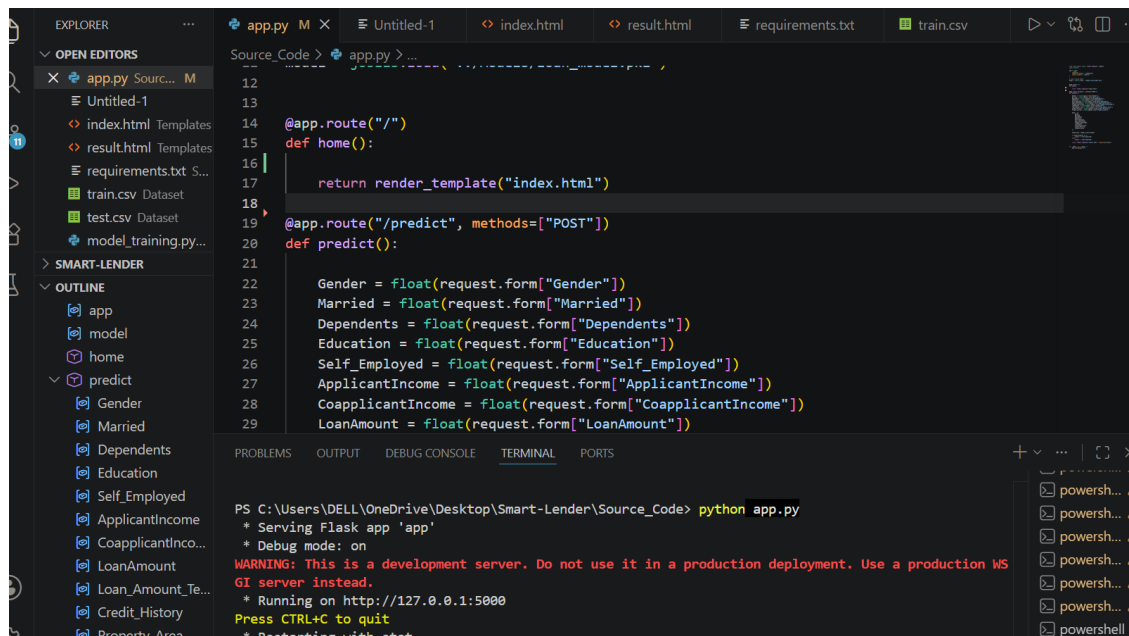
```
(venv) C:\Projects\Smart-Lender> pip install -r requirements.txt
```

Verify Project Files

Ensure the following files and folders are available:

- Source_Code/
- Templates/
- Static/
- Dataset/
- Models/
- requirements.txt

- app.py



```

12
13
14 @app.route("/")
15 def home():
16
17     return render_template("index.html")
18
19 @app.route("/predict", methods=["POST"])
20 def predict():
21
22     Gender = float(request.form["Gender"])
23     Married = float(request.form["Married"])
24     Dependents = float(request.form["Dependents"])
25     Education = float(request.form["Education"])
26     Self_Employed = float(request.form["Self_Employed"])
27     ApplicantIncome = float(request.form["ApplicantIncome"])
28     CoapplicantIncome = float(request.form["CoapplicantIncome"])
29     LoanAmount = float(request.form["LoanAmount"])
  
```

```

PS C:\Users\DELL\OneDrive\Desktop\Smart-Lender\Source_Code> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
  
```

Important Notes

- The application is currently executed using the **Flask Development Server**.
- A Python virtual environment is recommended before installing dependencies.
- Ensure all required Python libraries are installed using requirements.txt.
- The trained Machine Learning model (loan_prediction_model.pkl) must be available in the **Models** folder.
- The application requires the Loan Prediction dataset for model training.

Exploring the Website's Web Pages:

Home / Generator Page:

Smart Lender - Loan Prediction

Gender (Male=1, Female=0)

Married (Yes=1, No=0)

Dependents

Education (Graduate=1, Not Graduate=0)

Self Employed (Yes=1, No=0)

Applicant Income

Coapplicant Income

Loan Amount

Loan Amount Term

Description: The single-page interface of Smart-Lender serves as both the landing page and the loan prediction portal. It features a clean, responsive, and user-friendly design developed using HTML, CSS, Bootstrap, and JavaScript. The homepage displays the Smart-Lender title, a brief project description, and a loan application form where users enter applicant details such as Gender, Marital Status, Education, Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, and Property Area. After submitting the form, the application processes the information through the trained Machine Learning model and displays the loan eligibility prediction (Loan Approved or Loan Rejected) on the same page in a clear and user-friendly format.

Input Section:

Married (Yes=1, No=0)

Dependents

Education (Graduate=1, Not Graduate=0)

Self Employed (Yes=1, No=0)

Applicant Income

Coapplicant Income

Loan Amount

Loan Amount Term

Credit History (1 or 0)

Property Area (Urban=2 Semiurban=1 Rural=0)

Description: A responsive loan application form serves as the primary interface for users to enter applicant details required for loan eligibility prediction. The form includes fields such as Gender, Marital Status, Education, Self-Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, and Property Area. After filling in the required information, users click the Predict Loan Status button to submit the application. The Flask

backend validates the input, processes the data using the trained Machine Learning model, and displays the prediction result (Loan Approved or Loan Rejected) on the same page in a clear and user-friendly format. During processing, the application briefly indicates that the prediction is being generated, providing a smooth user experience.

Results Section:

Smart Lender

Loan Approved

[Predict Another Loan](#)

Description:

After the user submits the loan application form, the prediction result is displayed dynamically on the same page. The application processes the applicant's information using the trained Machine Learning model and presents the loan eligibility status in a dedicated results section. The result panel clearly indicates whether the loan is **Approved** or **Rejected**, along with a brief message explaining the outcome. The interface is designed using **HTML, CSS, Bootstrap, and Flask templates**, ensuring a clean, responsive, and user-friendly experience across desktop and mobile devices. The prediction result is displayed immediately after processing, allowing users to quickly understand their loan eligibility.

Conclusion

Smart-Lender is a Machine Learning-based web application developed using **Python, Flask, HTML, CSS, Bootstrap, JavaScript, and Scikit-learn** to simplify the loan approval prediction process. The system analyzes applicant information such as income, education, employment status, credit history, and loan amount to accurately predict whether a loan application is likely to be approved or rejected. By integrating a trained Machine Learning model with a responsive Flask web interface, the application provides quick, reliable, and user-friendly loan eligibility predictions.

The project successfully demonstrates the practical application of Machine Learning in the banking and financial sector by improving decision-making efficiency and reducing manual evaluation efforts. The modular architecture, clean user interface, and organized project structure make the system easy to maintain and extend. In the future, the application can be enhanced by integrating **user authentication, database support (MySQL or MongoDB), cloud deployment, loan history tracking, advanced Machine Learning models, and real-time banking API integration** to create a more scalable and intelligent loan management system.